



L LOVELY
P ROFESSIONAL
U NIVERSITY

Intelligent Curriculum & Attendance Management System

A project reports

Submitted in partial fulfilment of the requirements for the award of
degree of

**Bachelor Of Technology
(Computer Science and Engineering)**

Prepared by:

Name: Bijay Kumar Mishra

Registration No: 12200922

Underguidance of:

Name: Lovely Professional University

School of Computer Science and Engineering, Lovely
Professional University

Punjab, India

DECLARATION

I hereby declare that the project report entitled “**Intelligent Curriculum & Attendance Management System**” is a bonafide record of the work carried out by me, **Bijay Kumar Mishra**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at **Lovely Professional University, Punjab, India**.

I further declare that this project work is original and has been completed by me under proper guidance. It has not been submitted, either wholly or partially, to any other university or institution for the award of any degree or diploma.

All information, data, and references used in this project have been duly acknowledged.

Date: 2026

Place: Punjab, India

Signature of Student: bijaymsra

Name: Bijay Kumar Mishra

Registration No.: 12200922

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Lovely Professional University, Punjab, and the School of Computer Science and Engineering for providing the opportunity to carry out this project titled **“Intelligent Curriculum & Attendance Management System.”**

I am highly thankful to my project supervisor and faculty members for their continuous guidance, valuable suggestions, and constant encouragement during the development of this project. Their support and constructive feedback played a significant role in the successful completion of this work.

I would also like to thank my friends and peers for their support, helpful discussions, and motivation throughout the course of this project.

I would like to acknowledge the open-source community and the developers behind technologies such as React, Spring Boot, Python, Docker, Kubernetes, MySQL, Ollama, and AWS. Their tools, resources, and documentation greatly assisted in building this cloud-native and AI-integrated system.

Finally, I express my heartfelt gratitude to my family for their continuous encouragement, patience, and moral support during my academic journey.

ABSTRACT

Traditional attendance and curriculum management systems in educational institutions often rely on manual processes or monolithic applications. These systems usually lack scalability, real-time analytics, and intelligent support capabilities, which can lead to inefficiencies, inconsistent data management, and limited accessibility. Furthermore, many existing platforms do not integrate Artificial Intelligence to assist students and faculty with academic queries or system navigation.

To address these challenges, **Smart Curriculum AI (Attenza)** is proposed as a cloud-native attendance and curriculum management platform integrated with an AI-powered assistant. The system is designed using a microservices architecture to ensure modularity, scalability, and high availability. It includes a React-based frontend for user interaction, a Spring Boot backend for business logic and API management, a Python-based AI gateway service, and a MySQL database for persistent data storage.

The AI component integrates a locally hosted Large Language Model (**phi3**) through the **Ollama runtime**, enabling real-time academic assistance without relying on external AI APIs. This approach improves data privacy, reduces operational costs, and enhances response efficiency. The application stack is containerized using **Docker** and deployed using **Kubernetes (K3s)**, supporting dynamic scaling through the Horizontal Pod Autoscaler (HPA).

The platform is hosted on **AWS EC2** with an Elastic IP configuration and secured using HTTPS through **cert-manager** and **Let's Encrypt**. External traffic routing is handled by the **Traefik Ingress Controller**. Additional production practices such as resource limits, rolling updates, health monitoring endpoints, namespace isolation, and secure secret management are implemented to ensure system reliability.

Overall, Smart Curriculum AI demonstrates the integration of full-stack development, DevOps automation, Artificial Intelligence, and cloud infrastructure within a unified platform. The solution is scalable, secure, and suitable for real-world academic environments.

List of Contents

S.N	Title	Page
1	Declaration	2
2	Acknowledgment	3
3	Abstract	4
4	Chapter 1: Introduction	6-10
5	Chapter 2: Literature Review	10-14
6	Chapter 3: System Overview	14-20
7	Chapter 4: Technology Stack	20-23
8	Chapter 5: System Design	23-28
9	Chapter 6: Implementation	28-31
10	Chapter 7: Deployment & Infrastructure	31-34
11	Chapter 8: Testing & Validation	34-37
12	Chapter 9: Results & Screenshots	37-42
13	Chapter 10: Future Scope	42-44
14	Chapter 11: Conclusion	44-45
15	References	45-47
16	Appendix	47-52

Chapter 1: Introduction

1.1 Background of the Study

Educational institutions increasingly rely on digital platforms to manage academic activities such as attendance tracking, curriculum planning, student records, and performance monitoring. However, many institutions still depend on partially digitized or manual systems. Attendance is frequently maintained through paper registers or basic software tools that lack scalability, real-time analytics, and automation. These limitations often result in human errors, data inconsistencies, inefficiencies, and restricted accessibility.

Recent advancements in cloud computing, containerization, and artificial intelligence provide opportunities to modernize academic management systems. Cloud-native architectures enable applications to be scalable, resilient, and highly available, while container orchestration platforms such as Kubernetes support automated deployment, load balancing, and dynamic scaling. Despite these technological developments, many academic systems have not yet adopted production-grade cloud-native solutions for attendance and curriculum management.

Furthermore, modern educational platforms are expected to provide intelligent assistance to users. Students often require quick access to academic information and guidance related to coursework, while faculty members benefit from insights into attendance trends and student engagement. Traditional systems rarely integrate Artificial Intelligence capable of delivering contextual and real-time support.

The emergence of Large Language Models (LLMs) and locally deployable AI frameworks has enabled the integration of intelligent assistants directly into institutional platforms. By combining microservices architecture, cloud deployment strategies, secure authentication mechanisms, and AI-driven services, institutions can develop unified systems that improve efficiency while maintaining scalability and data privacy.

Therefore, there is a growing need for a cloud-native attendance and curriculum management platform integrated with Artificial Intelligence. This project focuses on designing and implementing such a system using modern full-stack development

practices, DevOps automation, containerization, and Kubernetes orchestration to create a production-ready solution suitable for real-world academic environments.

1.2 Problem Statement

Educational institutions require efficient systems to manage attendance records, curriculum activities, user roles, and academic insights. However, many existing solutions face limitations in scalability, automation, integration, and intelligent support.

Traditional attendance systems often rely on manual processes or basic digital tools operating in isolated environments. These systems are prone to human error, proxy attendance, data inconsistencies, and delayed reporting. Additionally, they lack real-time monitoring and analytical capabilities that could assist faculty and administrators in evaluating student engagement.

Another major challenge is the use of monolithic architectures in many academic management platforms. Tightly coupled systems are difficult to maintain and scale. During peak academic periods, such as class sessions or administrative operations, these systems may experience performance issues or downtime. They also lack automated scaling and cloud-native deployment mechanisms required for modern institutional infrastructure.

Security and deployment practices are also frequently inadequate. In several cases, systems do not implement secure secret management, encrypted communication, or proper infrastructure isolation. These limitations increase the risk of data exposure and operational instability.

Moreover, most traditional platforms do not incorporate Artificial Intelligence to assist users. Students often require immediate support for academic queries or system navigation, while faculty members could benefit from insights derived from attendance patterns. Without AI integration, such systems remain limited to basic record management.

Therefore, the core problem addressed in this project is the absence of a scalable, secure, cloud-native, and AI-enabled attendance and curriculum management system capable of supporting real institutional workloads while providing intelligent user assistance.

1.3 Objectives of the Project

The primary objective of the **Smart Curriculum AI (Attenza)** system is to develop a scalable, secure, and intelligent cloud-native platform for managing attendance and curriculum activities in educational institutions. The project integrates modern full-stack development practices, DevOps automation, and Artificial Intelligence to overcome the limitations of conventional academic systems.

The key objectives of the project are:

1. To develop a role-based academic management platform supporting administrators, faculty members, and students with secure authentication and authorization mechanisms.
2. To implement an attendance management module that enables real-time attendance tracking, session management, QR-based participation, and analytical insights for monitoring student engagement.
3. To design a microservices-based architecture that separates frontend, backend, AI services, and database components to improve modularity and maintainability.
4. To integrate an AI-powered academic assistant using a locally hosted Large Language Model through Ollama, enabling intelligent responses while maintaining data privacy.
5. To containerize application components using Docker and orchestrate them through Kubernetes to achieve scalability and efficient resource management.
6. To implement Horizontal Pod Autoscaling (HPA) for automatic scaling of backend services based on resource utilization.
7. To deploy the system on AWS EC2 with secure domain configuration, Elastic IP allocation, and HTTPS communication using cert-manager and Let's Encrypt.
8. To implement production-grade practices such as secure secret management, resource limits, health monitoring probes, rolling updates, and automated deployment mechanisms.

9. To provide a scalable architecture that supports future enhancements such as multi-tenant deployment, observability integration, caching mechanisms, and CI/CD automation.

1.4 Scope of the Project

The **Smart Curriculum AI (Attenza)** project focuses on the design, development, and deployment of a cloud-native attendance and curriculum management system integrated with an AI-powered assistant. The platform is intended for educational institutions such as colleges and universities to improve academic administration and user interaction.

The system is implemented as a full-stack web application consisting of a React-based frontend, a Spring Boot backend for business logic, a Python-based AI gateway service, and a MySQL relational database for persistent storage. The platform supports multiple user roles including Administrator, Faculty, and Student with secure authentication and role-based access control.

Functionally, the platform enables institution registration, class creation, attendance session management, QR-based attendance recording, and analytical insights for monitoring student participation. The AI assistant provides contextual academic support using a locally deployed Large Language Model through Ollama, ensuring data privacy and minimizing reliance on external AI services.

From an infrastructure perspective, the system uses Docker for containerization and Kubernetes (K3s) for orchestration. The deployment follows production-grade practices including namespace isolation, resource allocation limits, health monitoring probes, rolling updates, horizontal scaling, and secure secret management. The application is deployed on AWS EC2 with Elastic IP configuration and HTTPS security implemented through cert-manager and Let's Encrypt, while external traffic routing is handled by the Traefik Ingress Controller.

The current implementation focuses on a single-institution deployment model and does not include advanced multi-tenant architecture. Full observability integration using tools such as Prometheus and Grafana, along with mobile applications and distributed database systems, are considered future enhancements.

Overall, the project aims to deliver a secure, scalable, and production-ready academic management platform that demonstrates modern cloud-native architecture combined with Artificial Intelligence capabilities in an institutional environment.

Chapter 2: Literature Review

2.1 Existing Attendance Systems

Attendance management has long been an essential part of academic administration in schools, colleges, and universities. Traditionally, attendance was recorded manually using paper-based registers maintained by faculty members. Although simple to implement, this approach is highly susceptible to human error, proxy attendance, data loss, and time inefficiency. Additionally, manual systems make it difficult to generate consolidated reports or analyze attendance patterns over time.

With the advancement of digital technologies, many institutions adopted basic electronic attendance systems. These systems typically operate as desktop applications or simple web platforms where faculty manually record attendance through an administrative interface. While digital storage improves data organization and reduces paper dependency, many of these systems still lack scalability, automation, and real-time analytical capabilities.

More advanced approaches have introduced biometric devices, RFID cards, and fingerprint scanners to enhance attendance authenticity and reduce proxy attendance. Although these technologies improve accuracy, they require specialized hardware, increase implementation costs, and demand regular maintenance. Hardware-based solutions may also face limitations in remote or hybrid learning environments.

Some modern systems implement QR code-based attendance methods, allowing students to scan dynamically generated codes during class sessions. This approach simplifies attendance recording and reduces manual intervention. However, many of these systems remain monolithic in design and are not deployed using scalable cloud-native infrastructure, which can lead to performance issues during high usage.

Furthermore, most existing attendance platforms focus primarily on record management and reporting. They rarely incorporate intelligent analytics or Artificial

Intelligence features to assist students or faculty. As a result, users receive limited academic insights or automated support.

In summary, although attendance systems have evolved from manual registers to digital and hardware-assisted solutions, many still lack scalability, intelligent functionality, and modern cloud-based deployment. These limitations highlight the need for an advanced academic management platform that integrates cloud-native architecture and AI-driven capabilities.

2.2 Limitations of Current Solutions

Despite technological improvements, many current attendance and academic management systems continue to face several limitations that reduce their effectiveness in modern educational environments.

One major limitation is the use of monolithic system architecture. Many platforms are developed as single, tightly coupled applications where frontend, backend, and database components operate within the same deployment unit. Such architectures are difficult to maintain and scale, and performance may degrade significantly during periods of high user activity.

Scalability is another common challenge. Many existing systems operate on single-server environments without automated load balancing or horizontal scaling mechanisms. As the number of users increases, system performance may decline, resulting in slower response times or service interruptions.

Security concerns also remain significant. Several systems do not implement secure secret management, encrypted communication channels, or strong authentication and authorization mechanisms. These weaknesses increase the risk of unauthorized access and exposure of sensitive academic data.

Additionally, some platforms rely heavily on hardware-based attendance systems such as biometric scanners or RFID devices. While these technologies improve verification accuracy, they increase deployment costs and reduce flexibility in remote or hybrid learning scenarios.

Another important limitation is the lack of Artificial Intelligence integration. Most existing platforms function primarily as record-keeping tools without offering intelligent assistance or predictive insights. Students cannot obtain contextual academic guidance, and administrators have limited analytical support for decision-making.

Furthermore, many systems are not built using cloud-native technologies. They often lack containerization, automated deployment pipelines, health monitoring mechanisms, and infrastructure isolation. This limits system reliability, scalability, and long-term maintainability.

These limitations demonstrate the need for a modern attendance and curriculum management platform that incorporates microservices architecture, cloud-native deployment practices, and AI-powered user assistance.

2.3 Need for Cloud-Native Solutions

The increasing demand for scalable and reliable digital systems has highlighted the importance of cloud-native architectures in modern software development. Educational institutions that rely on digital platforms for academic operations require systems capable of handling dynamic workloads, maintaining high availability, and ensuring secure data management.

Cloud-native solutions are specifically designed to leverage cloud infrastructure through technologies such as microservices architecture, containerization, and orchestration platforms like Kubernetes. This architectural approach enables independent deployment and scaling of application components, improving flexibility and maintainability.

In the context of academic management systems, cloud-native architecture allows services such as the frontend interface, backend logic, AI modules, and database systems to operate independently while communicating through well-defined interfaces. This separation of components enhances system reliability and simplifies maintenance.

One of the major advantages of cloud-native systems is horizontal scalability. Container orchestration platforms can automatically adjust system resources based on performance metrics such as CPU and memory utilization. This allows the system to

handle peak workloads efficiently while optimizing resource usage during periods of low demand.

High availability and fault tolerance are also key benefits. Cloud-native platforms support rolling updates, automated recovery mechanisms, and continuous health monitoring through liveness and readiness probes. These capabilities help maintain uninterrupted service even when individual components fail.

Security is strengthened through encrypted communication, controlled resource allocation, and secure secret management practices. Additionally, cloud-native development supports continuous integration and continuous deployment (CI/CD), enabling faster updates and improved system reliability.

Given these advantages, adopting a cloud-native architecture is essential for building a modern academic management platform that is scalable, resilient, and suitable for real-world institutional requirements.

2.4 AI in Educational Systems

Artificial Intelligence has become an important technological advancement in the field of education. AI-driven systems are increasingly used to improve learning experiences, automate administrative tasks, and generate valuable insights from institutional data.

One significant application of AI in education is intelligent tutoring systems. These systems analyze student learning behavior and provide personalized recommendations to improve understanding and academic performance. By adapting to individual learning patterns, AI tools support personalized and self-paced learning.

AI is also used in academic analytics to examine large datasets and identify patterns related to attendance, engagement, and performance. Such insights enable administrators and faculty members to make informed decisions and implement strategies that improve student outcomes.

Another important advancement is the use of Natural Language Processing (NLP) in conversational AI systems. Modern Large Language Models enable the development of intelligent assistants capable of answering student queries in real time. These

assistants can provide explanations, assist with system navigation, and offer general academic guidance.

Despite these advancements, many traditional attendance and curriculum management systems do not integrate AI capabilities. They primarily function as administrative tools without offering intelligent support or automated insights.

Integrating Artificial Intelligence into academic management platforms can significantly enhance user experience and operational efficiency. By combining AI technologies with cloud-native infrastructure and secure deployment practices, institutions can develop intelligent systems capable of supporting both administrative processes and student learning needs.

Therefore, incorporating AI into educational platforms represents an important step toward building modern, adaptive, and intelligent academic management systems.

Chapter 3: System Overview

3.1 Proposed Solution

To overcome the limitations identified in existing attendance and curriculum management systems, the proposed solution is the development of **Smart Curriculum AI (Attenza)**, a scalable cloud-native academic management platform integrated with an Artificial Intelligence assistant.

The system is designed using a **microservices-based architecture** to ensure modularity, scalability, and maintainability. Instead of a monolithic structure, the application is divided into independent services including a React-based frontend for user interaction, a Spring Boot backend for business logic and API management, a Python-based AI gateway for handling AI queries, and a MySQL database for persistent data storage.

The platform supports three primary user roles: **Administrator, Faculty, and Student**. Administrators manage institutional data, users, classes, and analytics. Faculty members can initiate attendance sessions, monitor student participation, and access attendance reports. Students can join classes, mark attendance through QR-based mechanisms, and interact with the AI assistant for academic queries.

A key feature of the system is the integration of an AI-powered assistant using a locally deployed **Large Language Model (phi3)** through the **Ollama runtime**. Hosting the AI model locally improves data privacy, reduces dependency on external AI services, and minimizes operational costs. The AI module is implemented as a separate microservice to allow independent scaling and processing.

To ensure portability and scalability, all services are containerized using **Docker** and orchestrated using **Kubernetes (K3s)**. This infrastructure supports automated deployment, rolling updates, and horizontal scaling through the **Horizontal Pod Autoscaler (HPA)**. The system is deployed on **AWS EC2**, configured with an Elastic IP address and secured with HTTPS using **cert-manager** and **Let's Encrypt**, while traffic routing is handled through the **Traefik Ingress Controller**.

By combining cloud-native architecture, DevOps automation, and AI integration, the proposed solution provides a secure, scalable, and intelligent platform suitable for modern academic environments.

3.2 System Architecture

The **Smart Curriculum AI (Attenza)** platform follows a cloud-native, microservices-based architecture designed to ensure scalability, modularity, and reliability. The architecture is organized into multiple layers, each responsible for specific system functionalities.

The **Client Layer** represents end users accessing the system through web browsers. The user interface is developed using React and served via Nginx. This layer manages user interaction, dashboard rendering, and communication with backend APIs.

The **Ingress and Routing Layer** is handled by the Traefik Ingress Controller within the Kubernetes cluster. It manages external traffic and routes requests to appropriate services based on URL paths. HTTPS communication is enforced using TLS certificates issued by cert-manager and Let's Encrypt.

The **Application Services Layer** contains the Spring Boot backend, which implements core system logic including authentication, role-based access control, class management, attendance tracking, and REST API services. The backend communicates with the database to manage institutional and attendance data.

The **AI Service Layer** consists of a Python-based microservice responsible for processing AI queries. This service acts as a gateway between the frontend application and the locally deployed Large Language Model running through Ollama. Isolating the AI service ensures independent scaling and improved system modularity.

The **Data Persistence Layer** includes the MySQL database deployed within the Kubernetes cluster. It stores user information, attendance records, class details, and other institutional data. Database credentials are securely managed using Kubernetes Secrets.

All services are containerized and deployed as Kubernetes deployments, enabling automated recovery, load balancing, resource allocation, and horizontal scaling. This architecture ensures high availability, fault tolerance, and efficient resource utilization.

3.3 High-Level Architecture Diagram

The high-level architecture of **Smart Curriculum AI (Attenza)** is illustrated in Figure 3.3 and shows the overall flow of user requests from external access to internal services deployed within the Kubernetes environment.

Users access the platform through web browsers or mobile devices via the Internet. The system domain is mapped to an **Elastic IP address** associated with an **AWS EC2 instance**, which hosts a Kubernetes cluster responsible for managing all application services within a dedicated namespace.

Incoming traffic is handled by the **Traefik Ingress Controller**, which routes requests based on URL paths. Requests to the root path are directed to the **React frontend service**, API requests under /api are forwarded to the **Spring Boot backend**, and AI-related requests under /ai are routed to the **Python AI backend service**.

Secure communication is ensured through **HTTPS encryption**, automatically managed by **cert-manager** integrated with **Let's Encrypt**.

Within the Kubernetes cluster, the system consists of several independent services:

- **React Frontend (Nginx)** – Serves the static production build of the user interface.

- **Spring Boot Backend (Java 17)** – Implements REST APIs, authentication, role-based access control, and core business logic.
- **Python AI Backend** – Acts as a gateway for AI requests and communicates with the Ollama runtime.
- **Ollama Runtime** – Hosts the locally deployed **phi3 Large Language Model** for processing user queries.
- **MySQL Database** – Stores institutional data, user records, and attendance information.

The backend service is configured with a **Horizontal Pod Autoscaler (HPA)** that automatically adjusts the number of running pods based on CPU and memory usage. Additional production features such as health probes, rolling updates, and secure secret management further enhance system reliability and maintainability.

Overall, the architecture represents a **cloud-native, microservices-based deployment model** designed to provide scalability, security, and high availability for real-world academic environments.

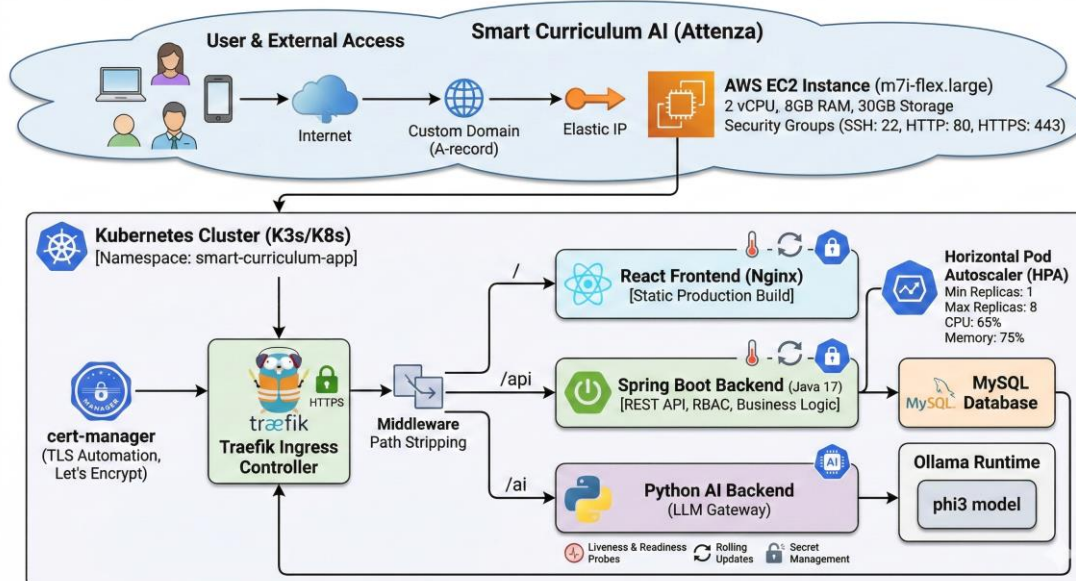


Figure 3.3: High-Level System Architecture of Smart Curriculum AI (Attenza)

3.4 Microservices Architecture

The Smart Curriculum AI platform adopts a **microservices architecture** to improve scalability, maintainability, and service isolation. In this approach, system functionality is divided into independent services that communicate through REST APIs within the Kubernetes cluster.

The **Frontend Service** is developed using React and served through Nginx as a static production build. It manages user interaction, dashboard rendering, and communication with backend APIs.

The **Backend Service**, implemented using Spring Boot, handles core business logic including authentication, class management, attendance tracking, and analytics generation. This service interacts directly with the MySQL database to manage persistent data.

The **AI Gateway Service** is implemented using Python and acts as an intermediary between the frontend and the locally hosted AI model. It processes user queries and forwards them to the Ollama runtime for response generation.

The **Ollama Runtime Service** hosts the phi3 Large Language Model within the Kubernetes cluster. By deploying the model locally, the system ensures improved data privacy and reduced reliance on external AI APIs.

The **Database Service** uses MySQL to store institutional records, user information, class schedules, and attendance data. Database credentials are securely managed through Kubernetes Secrets.

All services communicate internally using Kubernetes service discovery mechanisms, while external access is managed through the Ingress controller.

The microservices architecture provides several advantages including independent service scaling, improved fault isolation, easier maintenance, and faster system updates.

3.5 Deployment Architecture (AWS)

The deployment architecture of Smart Curriculum AI is designed to ensure secure and scalable cloud hosting using **Amazon Web Services (AWS)**.

The system is deployed on an **AWS EC2 instance**, which hosts a Kubernetes cluster responsible for managing all containerized services. The instance provides computing resources required for application services, database operations, and AI model execution.

A **static Elastic IP address** is assigned to the EC2 instance to allow stable external access. A custom domain name is mapped to this IP address, enabling users to access the platform through a user-friendly URL.

Network security is managed through **AWS Security Groups**, which allow inbound traffic only on essential ports such as SSH (22), HTTP (80), and HTTPS (443). All other ports remain restricted to enhance security.

Within the Kubernetes cluster, multiple services are deployed including the frontend, backend, AI service, Ollama runtime, and MySQL database. Each component operates in isolated containers and communicates through internal cluster networking.

External traffic is managed through the **Traefik Ingress Controller**, which routes requests to the appropriate services based on URL paths. HTTPS encryption is enforced

using **cert-manager** integrated with **Let's Encrypt**, enabling automatic certificate issuance and renewal.

The deployment architecture also supports **Horizontal Pod Autoscaling**, enabling backend services to automatically scale based on system load. Additional production practices such as rolling updates, health monitoring, and secure secret management ensure system reliability.

By combining AWS cloud infrastructure with Kubernetes orchestration, the system achieves a robust and scalable deployment model suitable for real-world academic environments.

Chapter 4: Technology Stack

4.1 Frontend Technologies

The frontend of **Smart Curriculum AI (Attenza)** is designed to provide a responsive and interactive user interface that enables users to access system functionalities efficiently. The frontend layer handles user interaction, dashboard rendering, and communication with backend services through REST APIs.

React is used as the primary frontend framework. React enables the development of dynamic single-page applications using a component-based architecture. This approach allows the interface to be divided into reusable components such as login forms, dashboards, navigation panels, attendance modules, and the AI assistant interface. The use of a virtual DOM improves rendering efficiency and application performance.

Tailwind CSS is used for styling the user interface. It is a utility-first CSS framework that allows rapid development of responsive layouts using predefined classes. Tailwind helps maintain consistent design across the platform while minimizing the need for extensive custom CSS.

After building the frontend application, the production bundle is served using **Nginx** inside a Docker container. Nginx acts as a lightweight web server responsible for delivering static files such as HTML, CSS, and JavaScript to users efficiently.

The frontend communicates with backend services through HTTP-based API calls routed via the Kubernetes ingress configuration. This separation ensures modular development and flexibility for future system enhancements.

4.2 Backend Technologies

The backend of the system is implemented using **Spring Boot with Java 17**, which provides a robust and scalable framework for building enterprise-level applications. The backend layer manages business logic, authentication, data processing, and communication between system components.

Spring Boot simplifies application development through built-in configurations, dependency management, and embedded server capabilities. It is used to develop RESTful APIs that support core functionalities such as user management, attendance tracking, class administration, and analytics generation.

Spring Security is integrated to enforce authentication and authorization mechanisms. It implements role-based access control (RBAC) to ensure that different user roles—Administrator, Faculty, and Student—can access only permitted system resources.

Database interaction is managed using **Spring Data JPA**, which provides an Object-Relational Mapping (ORM) layer. This framework enables efficient database operations through repository interfaces, reducing the need for manual SQL queries while maintaining data integrity.

Additionally, **Spring Boot Actuator** is used to expose system health endpoints that support monitoring and Kubernetes health checks. These endpoints enable automated detection of service availability and improve system reliability in a production environment.

4.3 AI Integration Technologies

Artificial Intelligence is integrated into the system to provide an intelligent assistant capable of responding to academic queries and assisting users with system navigation.

The AI module is implemented as an independent microservice using **Python**. This service acts as a gateway between the frontend interface and the locally hosted Large

Language Model. When users submit queries through the AI chat interface, the request is forwarded to the Python backend for processing.

The system uses **Ollama** as the runtime environment for hosting the Large Language Model locally. Hosting the model internally ensures data privacy, reduces dependency on external AI APIs, and improves response latency.

The **phi3 Large Language Model** is used to generate contextual responses for user queries. The model processes natural language input and provides relevant academic or system-related responses.

Communication between the AI service and the model runtime occurs through REST-based API calls. Deploying the AI module as a separate microservice allows independent scaling and prevents AI workloads from affecting the performance of core application services.

4.4 Database Technologies

The system uses **MySQL** as the primary relational database management system for storing institutional and attendance-related data. MySQL is widely used in enterprise applications due to its reliability, performance, and structured data management capabilities.

The database maintains records related to users, institutions, classes, attendance sessions, and attendance submissions. The relational schema ensures proper data organization through primary keys, foreign keys, and constraints that enforce referential integrity.

Integration with the backend is implemented through **Spring Data JPA**, which enables object-relational mapping between Java entities and database tables. This approach simplifies data access and improves maintainability of the application code.

The MySQL database is deployed as a containerized service within the Kubernetes cluster. Persistent storage is configured to ensure that data remains intact even if containers restart. Database credentials are securely managed using Kubernetes Secrets.

4.5 DevOps & Cloud Tools

The system incorporates modern DevOps practices and cloud infrastructure tools to support automated deployment, scalability, and monitoring.

Docker is used to containerize all system components, including the frontend, backend, AI service, database, and model runtime. Containerization ensures environment consistency across development and production environments while simplifying deployment.

Kubernetes (K3s) is used as the container orchestration platform responsible for managing containerized workloads. It provides automated deployment, service discovery, rolling updates, resource management, and self-healing capabilities.

The backend service uses the **Horizontal Pod Autoscaler (HPA)** to automatically scale application instances based on resource utilization. This allows the system to maintain performance during peak usage periods.

Traffic routing within the cluster is managed using the **Traefik Ingress Controller**, which directs incoming requests to the appropriate services. Secure communication is enforced through HTTPS using **cert-manager** integrated with **Let's Encrypt**.

The platform is deployed on **AWS EC2**, which provides cloud-based infrastructure for hosting the Kubernetes cluster. Security groups restrict network access to essential ports, ensuring secure external connectivity.

Together, these DevOps and cloud technologies enable automated deployment, scalable infrastructure, and reliable system operation within a cloud-native environment.

Chapter 5: System Design

5.1 ER Diagram

The Entity–Relationship (ER) diagram of **Smart Curriculum AI (Attenza)** illustrates the logical structure of the database and the relationships between the primary entities used in the system. The design follows relational database normalization principles to maintain data integrity, minimize redundancy, and clearly define relationships between entities.

Figure 5.1 presents the complete ER model of the system. The system is organized around several core entities representing major components of academic management. The **Institution** entity serves as the top-level organizational unit and establishes relationships with entities such as departments, faculty members, students, and administrative users. Each **Department** belongs to an institution and manages subjects, faculty members, and student groups.

The **Faculty** entity represents instructors responsible for managing courses and attendance sessions. Faculty members are associated with departments and institutions, and their permissions are defined through a dedicated permission structure that enables role-based access control.

The **Subject** entity represents academic subjects offered within departments. Subjects are linked with faculty members and student groups through the **CourseOffering** entity, which defines the assignment of subjects to specific faculty members and class groups.

The **StudentGroup** entity represents a batch or academic group of students enrolled in specific subjects. Students belonging to these groups participate in classes and attendance sessions.

The **AttendanceSession** entity records individual class sessions initiated by faculty members. Each session is associated with a specific student group and subject. The **AttendanceSubmission** entity stores attendance records for each student participating in a session.

Additional entities such as **TimetableEntry**, **TimeSlot**, and **Room** are used to manage scheduling information for classes. These entities help organize academic sessions by defining when and where classes take place.

Overall, the ER model establishes clear relationships between institutions, academic departments, users, courses, and attendance records. The structured relational design ensures data consistency and supports efficient data retrieval within the system.

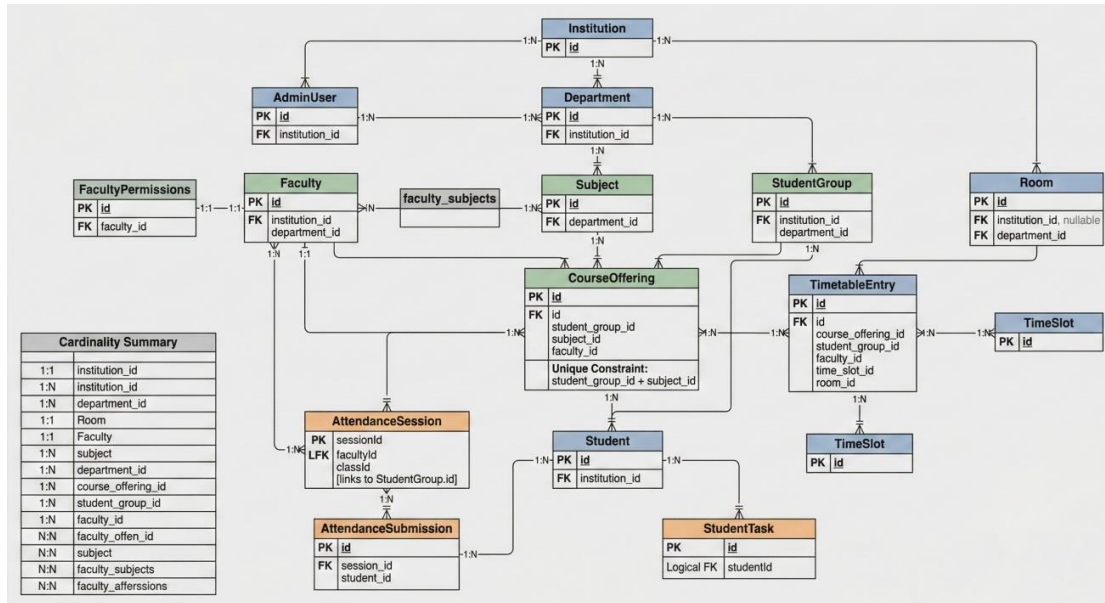


Figure 5.1 ER Diagram of Smart Curriculum AI System

5.2 Database Schema Design

The database schema of Smart Curriculum AI is derived from the ER model and implemented using **MySQL**. The schema follows normalization principles to ensure data consistency, reduce redundancy, and maintain referential integrity between entities.

Each table in the schema is defined using primary keys to uniquely identify records and foreign keys to establish relationships between related entities. Core tables include institutions, departments, faculty, students, subjects, course offerings, attendance sessions, and attendance submissions.

The **institution** table stores basic information about academic institutions and acts as the root entity of the system. Departments are associated with institutions through foreign key relationships. Similarly, faculty members and students are linked to institutions and departments to maintain organizational hierarchy.

The **course_offering** table acts as a junction entity connecting faculty members, subjects, and student groups. This structure ensures that each subject is assigned to the appropriate faculty member and class group.

Attendance management is implemented using two primary tables: **attendance_session** and **attendance_submission**. The attendance session table records individual class

sessions initiated by faculty members, while the attendance submission table stores attendance status for each student in a particular session.

Indexes are applied to frequently accessed attributes such as email identifiers, user IDs, and class IDs to improve query performance. This structured schema ensures efficient data retrieval and supports the operational requirements of the attendance management system.

5.3 API Design

The Smart Curriculum AI platform uses **RESTful API architecture** to enable communication between the frontend application, backend services, and AI module. The APIs are implemented using Spring Boot and follow standard HTTP methods to ensure consistency and scalability.

The API structure is organized into multiple functional modules, including authentication APIs, institution and user management APIs, course management APIs, attendance APIs, and AI interaction APIs.

Authentication endpoints manage user login and credential verification. Administrative APIs allow system administrators to manage institutional data, faculty members, and student accounts. Course management APIs enable faculty members to create course offerings and manage class schedules.

Attendance APIs support the creation of attendance sessions, submission of student attendance, and generation of attendance reports. These endpoints allow faculty members to initiate sessions and monitor student participation in real time.

AI-related interactions are handled through dedicated endpoints under the `/ai` path. These endpoints forward user queries to the AI backend service, which processes the request and returns responses generated by the Large Language Model.

All API responses follow a structured JSON format and use standard HTTP status codes to indicate request outcomes. This consistent API design ensures seamless communication between system components.

5.4 Authentication & Authorization Design

Security in Smart Curriculum AI is implemented using **role-based access control (RBAC)** to ensure that only authorized users can access protected resources. Authentication and authorization mechanisms are implemented using **Spring Security** within the backend service.

The authentication process verifies user identity using login credentials such as email and password. Passwords are securely stored using hashing mechanisms to prevent exposure of sensitive information.

After successful authentication, the system generates an authentication token that must be included in subsequent API requests. This token-based mechanism supports stateless communication and improves system scalability.

The platform defines three primary user roles: **Administrator**, **Faculty**, and **Student**. Each role is assigned specific permissions. Administrators manage institutional resources and user accounts, faculty members manage courses and attendance sessions, and students can view schedules, mark attendance, and interact with the AI assistant.

Authorization rules ensure that users can access only the endpoints permitted for their role. Requests are validated through security filters before reaching the application logic.

Additional security measures include HTTPS encryption, secure credential management through Kubernetes Secrets, and restricted access to sensitive endpoints. These practices ensure secure communication and protection of academic data.

5.5 AI Module Design

The AI module of Smart Curriculum AI is designed as an independent microservice responsible for processing user queries and generating intelligent responses. Separating the AI module from the main backend improves modularity, scalability, and maintainability.

The AI service is implemented using **Python** and acts as an intermediary between the frontend interface and the Large Language Model runtime. When a user submits a query through the AI chat interface, the request is routed to the AI backend service through the **/ai/chat** endpoint.

The AI backend forwards the request to the **Ollama runtime**, which hosts the **phi3 Large Language Model**. The model processes natural language input and generates context-aware responses relevant to academic queries.

The response generated by the model is returned to the AI backend and then forwarded to the frontend interface in structured JSON format. This architecture ensures efficient processing of AI requests while maintaining separation between core application logic and AI functionality.

Deploying the AI module as a separate microservice allows independent scaling based on usage patterns. The AI service communicates internally within the Kubernetes cluster, ensuring secure interaction with other system components.

This design enables the platform to provide intelligent academic assistance while maintaining performance, scalability, and data privacy within a cloud-native infrastructure.

Chapter 6: Implementation

6.1 Frontend Implementation

The frontend of **Smart Curriculum AI (Attenza)** is implemented as a **Single Page Application (SPA)** using React. The frontend layer is responsible for rendering the user interface, managing user interactions, and communicating with backend services through REST APIs.

The application follows a modular structure consisting of reusable components, pages, service modules, and routing configurations. Reusable components such as login forms, navigation menus, dashboards, attendance panels, and the AI chat interface improve maintainability and support scalable UI development.

Role-based routing is implemented to ensure that users access only the interfaces relevant to their permissions. After authentication, users are redirected to their respective dashboards, including **Administrator**, **Faculty**, or **Student** views.

The frontend communicates with backend APIs through HTTP requests. All requests include authentication tokens in the request headers to ensure secure communication with protected endpoints.

Responsive user interfaces are implemented using **Tailwind CSS**, enabling consistent styling and adaptive layouts across devices. After development, the application is compiled into a production build and served through **Nginx** within a Docker container, ensuring efficient delivery of static resources.

6.2 Backend Implementation

The backend of the system is developed using **Spring Boot (Java 17)** and functions as the core processing layer of the platform. It manages business logic, authentication mechanisms, database interactions, and communication with external services.

The backend follows a layered architecture consisting of the **Controller**, **Service**, and **Repository** layers. The Controller layer handles HTTP requests and routes them to appropriate service methods. The Service layer implements core business logic such as attendance session creation, class management, and report generation. The Repository layer interacts with the database using **Spring Data JPA**, enabling efficient data persistence and retrieval.

Security is implemented using **Spring Security**, which enforces authentication and role-based authorization. The system restricts access to protected APIs based on user roles, ensuring that administrators, faculty members, and students can only access their permitted functionalities.

Attendance functionality is implemented through structured workflows. Faculty members initiate attendance sessions, students submit attendance records, and the backend processes and stores these records in the database. Attendance reports are generated dynamically through database queries.

To support system reliability, **Spring Boot Actuator** is integrated to expose health monitoring endpoints used by Kubernetes probes. This enables automated detection of service availability and improves operational stability.

6.3 AI Backend Implementation

The AI backend is implemented as a separate microservice using **Python** to process user queries and interact with the locally hosted Large Language Model. This

microservice architecture ensures modularity and prevents AI processing tasks from affecting the performance of the main backend service.

The AI service receives user queries through the `/ai/chat` endpoint. After validating the request, the service forwards the query to the **Ollama runtime**, which hosts the **phi3 Large Language Model**.

The model processes the natural language input and generates an appropriate response. The AI backend then formats the response and returns it to the frontend interface in JSON format.

Environment variables are used to configure parameters such as the model endpoint and model name, allowing flexible configuration without modifying application code.

By isolating the AI functionality into a separate service, the system supports independent scaling and efficient handling of AI workloads while maintaining stable core system performance.

6.4 Docker Implementation

Docker is used to containerize all system components, ensuring consistency across development and deployment environments. Each major service—including the frontend, backend, AI service, model runtime, and database—is packaged into its own container.

The backend service uses a multi-stage Docker build process to compile the application and generate a lightweight runtime image. Similarly, the frontend application is built into a static bundle and served through an Nginx container.

The AI backend container installs required Python dependencies and executes the AI service. The Ollama runtime and MySQL database are also deployed as containerized services within the system environment.

Containerization simplifies application deployment, improves portability, and enables efficient integration with orchestration platforms such as Kubernetes. Docker images are versioned and deployed consistently across environments, ensuring reproducible builds and stable deployments.

6.5 Kubernetes Implementation7.5 Security Practices

Kubernetes is used as the orchestration platform for managing containerized services of the Smart Curriculum AI system. The platform is deployed using **K3s**, a lightweight Kubernetes distribution suitable for cloud-based environments.

Each service is deployed using **Kubernetes Deployments**, which manage container lifecycle, replication, and automated recovery. If a container fails, Kubernetes automatically recreates it, ensuring high availability.

Services within the cluster communicate through internal networking using **ClusterIP services**, while external traffic is routed through the **Traefik Ingress Controller**. Ingress rules map URL paths to the corresponding services, enabling structured routing of frontend, API, and AI requests.

To ensure secure communication, **cert-manager** automatically provisions TLS certificates through **Let's Encrypt**, enabling HTTPS encryption for all external traffic.

The backend service is configured with a **Horizontal Pod Autoscaler (HPA)**, which dynamically adjusts the number of running pods based on CPU and memory usage. This allows the system to maintain performance during peak usage periods.

Additional configurations such as resource limits, liveness and readiness probes, namespace isolation, and Kubernetes Secrets for credential management enhance system reliability and security.

Through container orchestration, automated scaling, and secure routing, the Kubernetes implementation provides a robust foundation for deploying the Smart Curriculum AI platform in a cloud-native environment.

Chapter 7: Deployment & Infrastructure

7.1 AWS EC2 Setup

The **Smart Curriculum AI (Attenza)** platform is deployed on **Amazon Web Services (AWS)** using an **Elastic Compute Cloud (EC2)** instance as the primary infrastructure component. The EC2 instance acts as the host environment for the Kubernetes cluster that manages all containerized application services.

An Ubuntu-based server environment is used to run the Kubernetes distribution (**K3s**), which orchestrates the frontend, backend, AI service, database, and supporting components. The selected instance configuration provides adequate computing resources to support application workloads, container operations, and AI model execution.

Secure access to the EC2 instance is established using **SSH key pairs**, allowing administrators to manage the server remotely. The cloud-based deployment ensures reliability, scalability, and availability of the application infrastructure.

7.2 Elastic IP & Domain Configuration

To ensure reliable external connectivity, an **Elastic IP address** is assigned to the EC2 instance. This static public IP remains unchanged even if the instance is restarted, providing a stable access point for the deployed system.

A **custom domain name** is configured and linked to the Elastic IP through DNS records using Cloudflare, as shown in **Figure 7.2**. This configuration maps the main domain and its subdomains to the server’s public IP address, allowing users to access the platform through a readable web address instead of a direct IP.

The domain mapping improves system accessibility and also supports secure HTTPS communication through certificate management services.

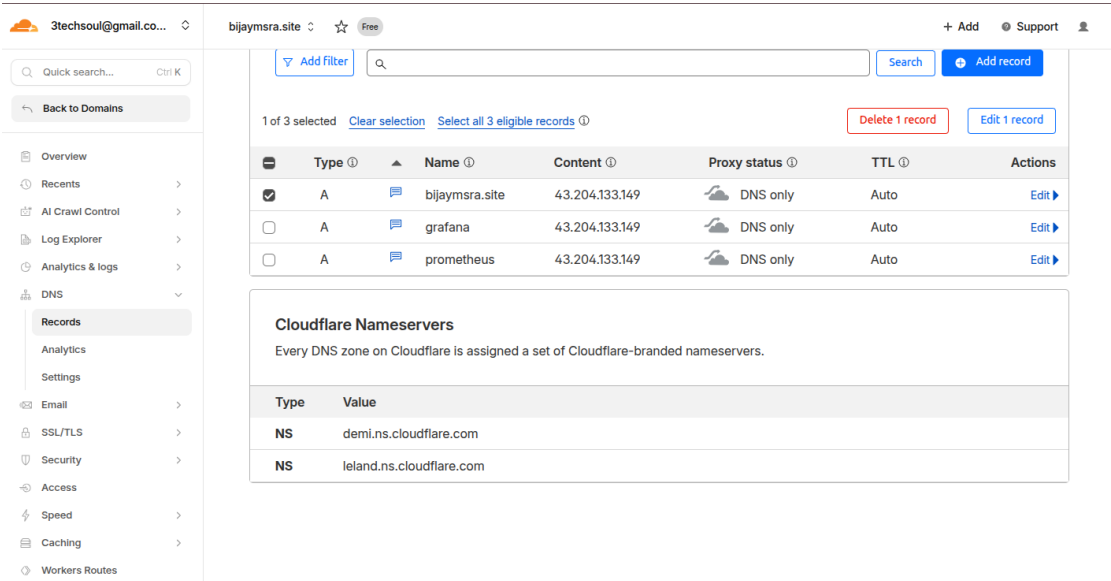


Figure 7.2: Cloudflare DNS Records Mapping Domain to Elastic IP

7.3 Ingress & TLS Configuration

External traffic to the Kubernetes cluster is managed using the **Traefik Ingress Controller**. The Ingress configuration defines routing rules that direct incoming requests to appropriate services within the cluster.

Requests directed to the root path are routed to the frontend service, while API requests and AI queries are forwarded to the backend and AI services respectively. This routing mechanism ensures structured and secure communication between external users and internal services.

To secure data transmission, **TLS encryption** is implemented using **cert-manager** integrated with **Let's Encrypt**. The system automatically generates and renews SSL certificates, ensuring encrypted HTTPS communication between users and the platform.

This automated certificate management reduces manual configuration and improves overall security of the deployment.

7.4 Horizontal Pod Autoscaler

To maintain system performance during varying workloads, the backend service is configured with a **Horizontal Pod Autoscaler (HPA)** in Kubernetes.

The HPA monitors resource usage metrics such as CPU utilization and automatically adjusts the number of running pods based on predefined thresholds. During high user activity, additional pods are created to handle increased load, while unnecessary pods are removed when demand decreases.

This automated scaling mechanism improves resource efficiency, maintains system responsiveness, and reduces manual infrastructure management.

7.5 Security Practices

Several security measures are implemented to ensure safe deployment and operation of the platform.

Network-level security is managed through **AWS Security Groups**, which restrict inbound traffic to essential ports such as SSH (22), HTTP (80), and HTTPS (443). This prevents unauthorized access to the infrastructure.

Sensitive information such as database credentials and application configuration values are stored using **Kubernetes Secrets** rather than being hardcoded in configuration files. This approach protects confidential data within the cluster environment.

Additionally, all external communication with the system is encrypted using **HTTPS**, ensuring secure data transmission between users and the application.

Combined with role-based access control implemented at the application level, these infrastructure-level practices provide a secure and reliable deployment environment for the Smart Curriculum AI platform.

Chapter 8: Testing & Validation

8.1 Unit Testing

Unit testing is conducted to verify the correctness of individual components of the system in isolation. In **Smart Curriculum AI (Attenza)**, unit tests primarily focus on backend services, business logic methods, and repository operations.

The Spring Boot testing framework with **JUnit** is used to validate service-layer functionality, authentication mechanisms, attendance session creation, and data validation logic. These tests ensure that individual methods perform as expected before interacting with other modules.

For the AI backend, functional tests are executed to verify correct request–response communication with the **Ollama runtime**. Unit testing helps detect errors at an early stage, improving overall system reliability and maintainability.

8.2 API Testing

API testing is performed to ensure that REST endpoints operate correctly and return expected responses. Testing tools such as **Postman** and **curl** are used to validate API functionality.

Test scenarios include login authentication, role-based endpoint access, attendance session creation, attendance submission, and AI query processing. Each endpoint is evaluated for correct HTTP status codes, response structure, and error handling behavior.

For example, the authentication endpoint is tested to verify successful login responses for valid credentials and appropriate error responses for invalid inputs. This testing ensures that the system APIs comply with REST standards and function reliably under different request conditions.

8.3 Integration Testing

Integration testing verifies the interaction between different system components. This testing phase ensures that services communicate correctly within the microservices architecture.

Key integrations tested include communication between the **frontend and backend**, backend interaction with the **MySQL database**, and AI backend communication with the **Ollama runtime**. In addition, Kubernetes service-to-service communication and Ingress routing configurations are validated.

Typical test scenarios include user authentication followed by dashboard rendering, faculty initiating attendance sessions with students submitting attendance, and AI queries being processed through the full system pipeline. These tests confirm that all system components operate together as expected.

8.4 System Testing

System testing evaluates the complete platform as an integrated application. This stage verifies that both functional and non-functional requirements are satisfied.

Functional testing includes verifying institution registration, role-based dashboard access, attendance management workflows, AI assistant interaction, and secure HTTPS communication.

Non-functional testing focuses on aspects such as system availability, security enforcement, correct request routing, and monitoring visibility through tools such as Prometheus and Grafana.

The system is accessed through the configured domain to simulate real-world deployment conditions and validate the behavior of the complete application.

8.5 Performance Considerations

Performance evaluation focuses on system responsiveness, scalability, and resource utilization under varying workloads.

Several strategies are implemented to ensure optimal performance, including **Horizontal Pod Autoscaling (HPA)** in Kubernetes, resource requests and limits for containers, and database indexing for efficient query execution. Static frontend content is served through **Nginx**, improving response speed and reducing backend load.

System performance is monitored using **Prometheus** and **Grafana**, which provide real-time metrics such as CPU usage, memory consumption, and pod health status. These monitoring tools help analyze system behavior and detect potential performance issues.

Testing results indicate that the platform maintains stability under load conditions and automatically scales application services when resource utilization increases.

Validation Summary

The testing and validation process confirms that the Smart Curriculum AI platform meets functional requirements, enforces security policies, and maintains stable performance. The system successfully integrates AI services, supports automated scaling, and ensures secure communication through HTTPS.

Overall, the comprehensive testing approach demonstrates that the platform is reliable and suitable for deployment in real-world academic environments.

Chapter 9: Results & Screenshots

This chapter presents the functional results of the Smart Curriculum AI (Attenza) system. The screenshots demonstrate successful implementation of role-based dashboards, AI integration, and cloud-native deployment monitoring. Each interface validates system functionality and confirms real-world operational readiness.

9.1 Landing Page

The Landing Page serves as the public entry point of the Smart Curriculum AI platform. It provides users with an overview of the system and access to authentication features such as login and institution registration.

The interface is designed using React and Tailwind CSS to ensure a responsive and modern user experience. The page is served through Nginx and is accessible securely via HTTPS.

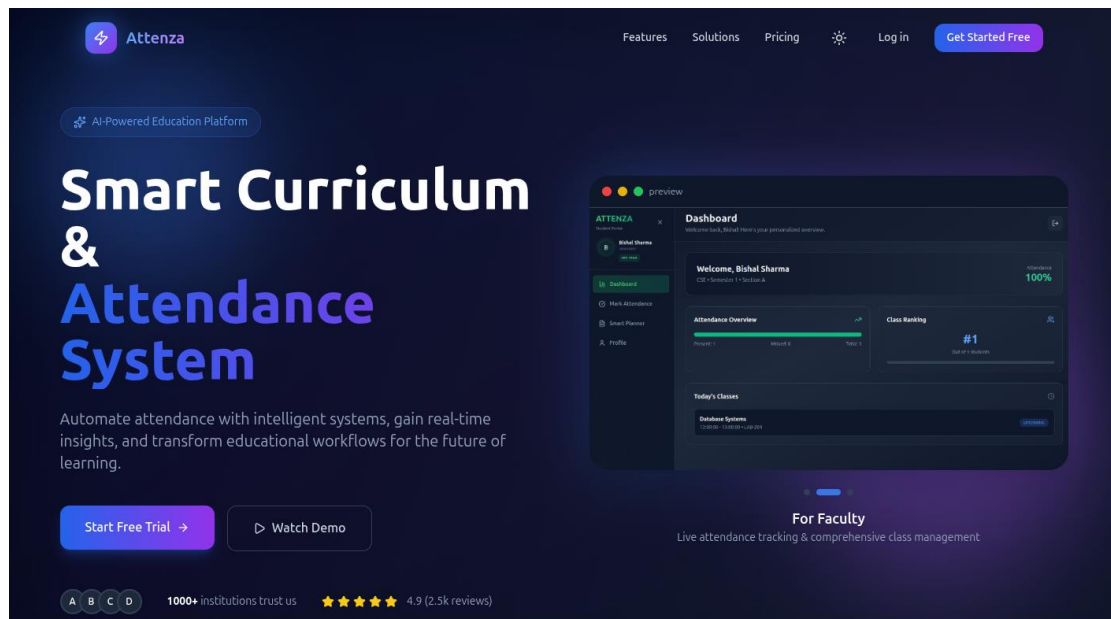


Figure 9.1: Landing Page of Smart Curriculum AI

The landing page confirms successful frontend deployment, domain configuration, and secure HTTPS access. It demonstrates responsive UI rendering and initial system accessibility from the public internet.

9.2 Admin Dashboard

The Admin Dashboard provides centralized control over institutional operations. Administrators can manage departments, faculty members, students, class groups, and system-wide analytics.

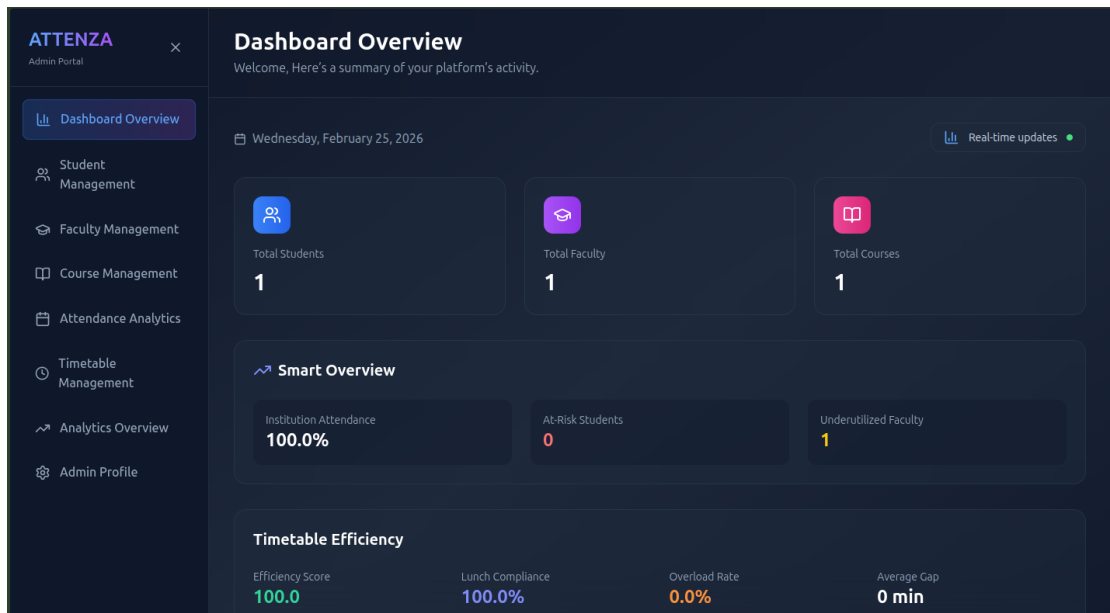


Figure 9.2: Administrator Dashboard Interface

The dashboard validates role-based access control (RBAC) implementation. Only users authenticated with administrator privileges can access this interface. The dashboard confirms successful backend API integration and secure data retrieval from the database.

9.3 Faculty Dashboard

The Faculty Dashboard enables instructors to manage course offerings, initiate attendance sessions, and monitor class participation.

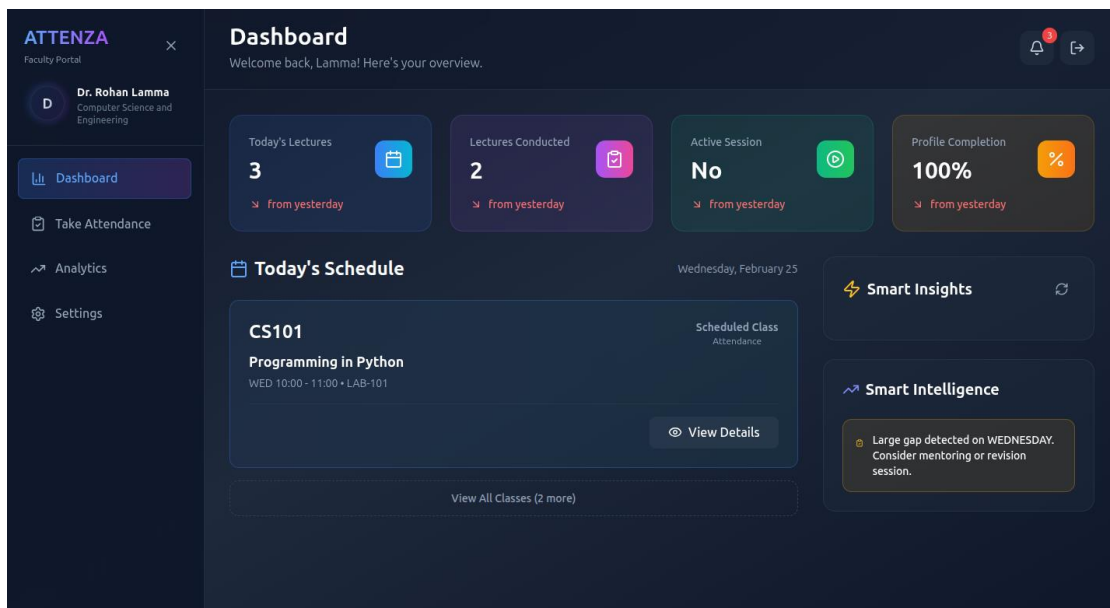


Figure 9.3: Faculty Dashboard Interface

This interface demonstrates the operational attendance workflow. Faculty members can create attendance sessions and generate QR codes for students. The dashboard confirms successful integration between frontend, backend services, and database operations.

9.4 Student Dashboard

The Student Dashboard allows students to view enrolled classes, submit attendance via QR scanning, track attendance history, and access academic features.

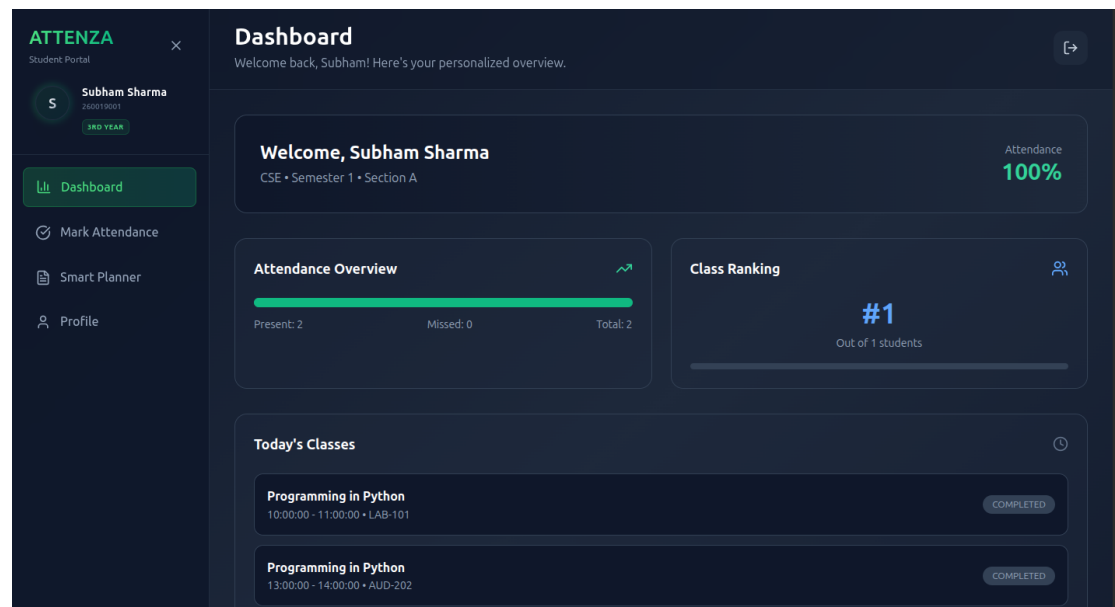


Figure 9.4: Student Dashboard Interface

The student interface validates user-specific data rendering and secure session handling. Attendance records displayed on this dashboard confirm accurate data persistence and retrieval.

9.5 AI Assistant Interface

The AI Assistant Interface provides students with real-time academic assistance using the locally deployed Large Language Model (phi3) integrated via Ollama.

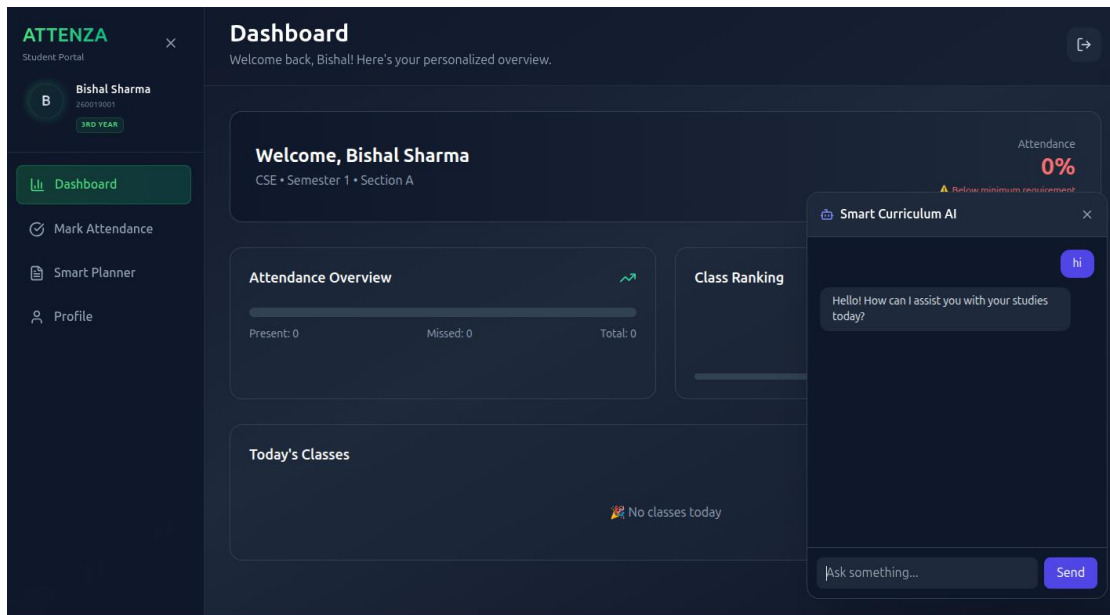


Figure 9.5: AI Assistant Chat Interface

This interface confirms successful AI microservice integration. Queries submitted by students are processed through the Python AI backend and forwarded to the Ollama runtime. The generated responses demonstrate functional LLM inference within the Kubernetes cluster.

This section validates:

- AI service routing via /ai endpoint
- Secure internal communication
- Contextual natural language response generation

9.6 Kubernetes Monitoring

Kubernetes monitoring ensures system stability, resource management, and scalability within the cloud-native environment.

```
ubuntu@ip-172-31-20-171:~/k8s$ kubectl get pod -n attenza
NAME                                READY   STATUS    RESTARTS   AGE
ai-backend-5b65f8977-zzhzb         1/1     Running   0           3m19s
backend-7b575bcb5c-t6h2d           1/1     Running   0           6m33s
frontend-55f8d667fb-8wzcb         1/1     Running   0           5m32s
mysql-0                             1/1     Running   0           6m50s
ollama-74cc78bc8-gvw5r             1/1     Running   0           5m24s
ubuntu@ip-172-31-20-171:~/k8s$ kubectl get hpa -n attenza
NAME           REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa    Deployment/backend        cpu: 1%/65%, memory: 57%/75%   1         8         1         57s
ubuntu@ip-172-31-20-171:~/k8s$
```

Figure 9.6: Kubernetes Cluster Monitoring

This screenshot validates successful container orchestration, pod management, and service deployment. It confirms that all microservices are running within the Kubernetes namespace and that the system is fully containerized.

Monitoring confirms:

- Pod health status
- Replica management
- Service availability

9.7 Prometheus and Grafana Dashboard

Prometheus and Grafana are integrated to monitor system performance and resource utilization in real time.

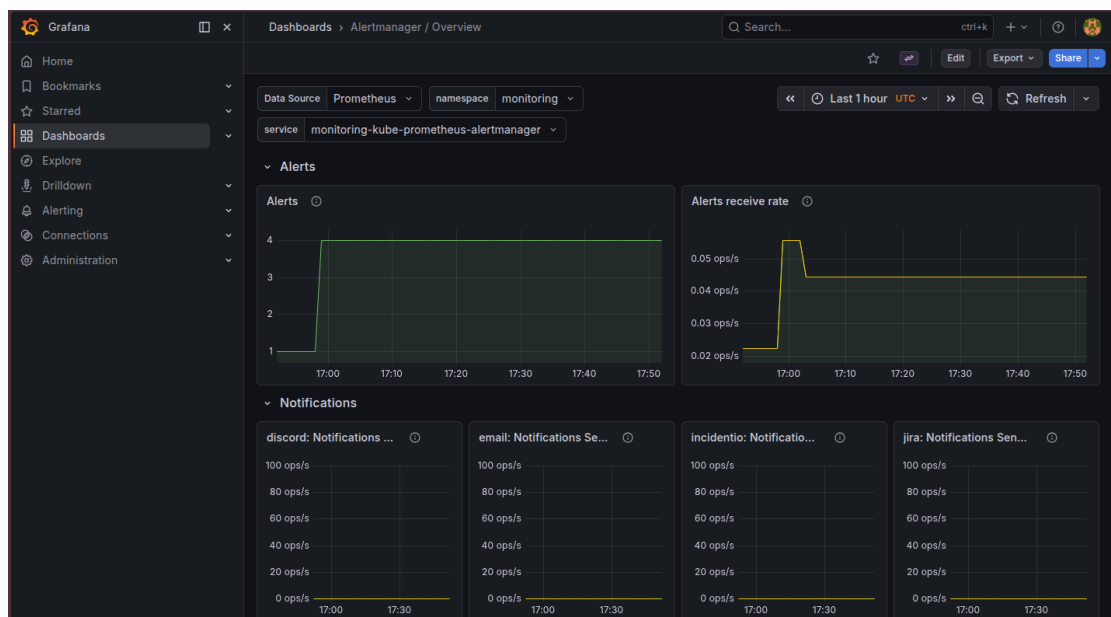


Figure 9.7: Grafana Dashboard Displaying System Metrics

The dashboard displays metrics such as CPU usage, JVM memory utilization, and backend performance indicators. These metrics confirm effective resource allocation, autoscaling behavior, and system stability under load.

Prometheus collects time-series metrics, while Grafana visualizes them for operational analysis. This integration validates production-grade observability and infrastructure reliability.

Chapter 10: Future Scope

The **Smart Curriculum AI (Attenza)** platform has been developed as a scalable cloud-native academic management system integrated with Artificial Intelligence. While the current implementation provides a production-ready solution, several enhancements can further improve scalability, functionality, and adaptability for institutional environments.

Multi-Tenant Architecture

The present system supports a structured institutional hierarchy; however, future versions can introduce full **multi-tenant architecture**. This would allow multiple institutions to operate independently on the same infrastructure while maintaining strict data isolation. Tenant-aware authentication and database segregation mechanisms could strengthen this capability.

Advanced Observability

Although monitoring is currently supported using **Prometheus and Grafana**, future improvements may include centralized logging and distributed tracing tools such as the **ELK Stack** and **Jaeger**. These tools would enhance system visibility, simplify debugging, and enable proactive incident detection.

Caching Mechanism

Introducing **Redis** as a distributed caching layer can improve system performance by reducing database load and accelerating access to frequently requested data. Caching would also enhance session handling and system responsiveness during peak workloads.

CI/CD Pipeline Improvements

The existing CI/CD workflow can be further expanded to support automated Docker image publishing, staging and production environment separation, automated rollback mechanisms, and container security scanning. These improvements would strengthen deployment automation and reliability.

AI Model Enhancement

Currently, the platform integrates the **phi3 model** through the Ollama runtime. Future improvements may include support for multiple AI models, fine-tuned academic

models, context-aware conversations, and optimized inference performance. These enhancements would improve the intelligence and adaptability of the AI assistant.

Mobile Application Integration

Developing dedicated **mobile applications for Android and iOS** can significantly enhance accessibility. Mobile support would allow students and faculty to access attendance systems, receive notifications, interact with the AI assistant, and monitor academic activities directly from mobile devices.

Advanced Analytics

Future versions of the system may incorporate advanced analytics features such as predictive attendance analysis, academic performance correlation, and customizable reporting dashboards. Machine learning algorithms could generate insights to support data-driven decision-making for administrators.

Enhanced Security

Additional security features such as **Multi-Factor Authentication (MFA)**, OAuth2-based authentication, audit logging, and API rate limiting can further strengthen system security and compliance with enterprise security standards.

Scalable Cloud Deployment

The current deployment uses a single-node Kubernetes environment. Future implementations may transition to **multi-node Kubernetes clusters** or managed cloud services to support large-scale deployments and improved fault tolerance.

Future Vision

The Smart Curriculum AI platform establishes a foundation for modern digital academic management. With continued improvements in DevOps automation, AI capabilities, and cloud scalability, the system can evolve into a comprehensive enterprise-level academic management platform while maintaining its modular microservices architecture.

Chapter 11: Conclusion

The **Smart Curriculum AI (Attenza)** project demonstrates the design and implementation of a cloud-native, scalable, and AI-enabled academic management platform. The system addresses limitations of traditional attendance and curriculum management solutions by integrating modern full-stack development, microservices architecture, DevOps practices, and intelligent assistance through a locally hosted Large Language Model.

The platform provides **role-based access control** for administrators, faculty members, and students, enabling structured and secure academic operations. Features such as dynamic attendance sessions, QR-based participation, institutional hierarchy management, and analytical dashboards improve efficiency in academic environments.

From a technical perspective, the system follows a **microservices-based architecture**. The frontend is developed using React and served through Nginx, while the backend is implemented with Spring Boot to handle REST APIs, authentication, and business logic. The AI module operates as an independent Python service connected to the Ollama runtime hosting the **phi3** model. Data persistence is maintained through a normalized **MySQL relational database**.

The deployment infrastructure follows **cloud-native best practices**. All services are containerized using Docker and orchestrated with Kubernetes (K3s). The system includes automated scaling through Horizontal Pod Autoscaling, health monitoring through liveness and readiness probes, namespace isolation, and secure secret management. External access is managed using the Traefik Ingress Controller with automated TLS certificates issued by cert-manager and Let's Encrypt. Hosting on AWS EC2 with an Elastic IP ensures stable and reliable deployment.

DevOps automation is supported through **GitHub Actions** for continuous integration and deployment, while **Prometheus and Grafana** provide monitoring and observability for system performance and health metrics.

Overall, the Smart Curriculum AI platform demonstrates the practical integration of cloud infrastructure, container orchestration, DevOps automation, and Artificial Intelligence within an academic management system. The modular architecture and scalable deployment model make the platform suitable for real-world institutional use and future expansion into enterprise-level academic solutions.

References

The following references were used in the design, development, and deployment of the Smart Curriculum AI (Attenza) platform. These include official documentation, technical guides, and standards relevant to cloud-native architecture, DevOps practices, database design, and AI integration.

1. Kubernetes Documentation. (n.d.). *Kubernetes Official Documentation*. Retrieved from <https://kubernetes.io/docs/>
2. Docker Documentation. (n.d.). *Docker Official Documentation*. Retrieved from <https://docs.docker.com/>
3. Spring Boot Documentation. (n.d.). *Spring Boot Reference Guide*. Retrieved from <https://docs.spring.io/spring-boot/docs/current/reference/html/>
4. Spring Security Documentation. (n.d.). *Spring Security Reference Guide*. Retrieved from <https://docs.spring.io/spring-security/reference/>
5. React Documentation. (n.d.). *React Official Documentation*. Retrieved from <https://react.dev/>
6. Tailwind CSS Documentation. (n.d.). *Tailwind CSS Official Documentation*. Retrieved from <https://tailwindcss.com/docs>
7. MySQL Documentation. (n.d.). *MySQL Reference Manual*. Retrieved from <https://dev.mysql.com/doc/>
8. AWS Documentation. (n.d.). *Amazon EC2 User Guide*. Retrieved from <https://docs.aws.amazon.com/ec2/>
9. Traefik Documentation. (n.d.). *Traefik Ingress Controller Documentation*. Retrieved from <https://doc.traefik.io/traefik/>
10. cert-manager Documentation. (n.d.). *cert-manager Official Documentation*. Retrieved from <https://cert-manager.io/docs/>
11. Let's Encrypt. (n.d.). *Let's Encrypt Documentation*. Retrieved from <https://letsencrypt.org/docs/>

12. Prometheus Documentation. (n.d.). *Prometheus Monitoring Documentation*. Retrieved from <https://prometheus.io/docs/>
13. Grafana Documentation. (n.d.). *Grafana Official Documentation*. Retrieved from <https://grafana.com/docs/>
14. GitHub Actions Documentation. (n.d.). *GitHub Actions CI/CD Documentation*. Retrieved from <https://docs.github.com/actions>
15. Ollama Documentation. (n.d.). *Ollama Model Runtime Documentation*. Retrieved from <https://ollama.com/docs>
16. Microsoft. (2023). *Introduction to Microservices Architecture*. Retrieved from <https://learn.microsoft.com/>
17. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
18. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software Architectures* (Doctoral dissertation, University of California, Irvine).

Appendix

The appendix section provides supporting technical artifacts and configuration files that were used in the development and deployment of Smart Curriculum AI (Attenza). These materials complement the implementation details described in previous chapters and provide deeper insight into the system's technical foundation.

Appendix A: Dockerfiles

This section contains representative Docker configuration files used to containerize system components.

A.1 Backend Dockerfile (Spring Boot)

```
# Build Stage
```

```
FROM maven:3.9-eclipse-temurin-17 AS build
```

```
WORKDIR /app
```

```
COPY pom.xml .
```

```
COPY src ./src
```

```
RUN mvn clean package -DskipTests
```

```
# Runtime Stage
```

```
FROM eclipse-temurin:17-jre
```

```
WORKDIR /app
```

```
COPY --from=build /app/target/backend.jar app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A.2 Frontend Dockerfile (React + Nginx)

```
# Build Stage
```

```
FROM node:18 AS build
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
# Production Stage
```

```
FROM nginx:alpine
```

```
COPY --from=build /app/build /usr/share/nginx/html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

A.3 AI Backend Dockerfile (Python)

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

Appendix B: Kubernetes YAML Files

This section provides representative Kubernetes configuration files used for deployment.

B.1 Namespace Configuration

apiVersion: v1

kind: Namespace

metadata:

name: attenza

B.2 Backend Deployment (Snippet)

apiVersion: apps/v1

kind: Deployment

metadata:

name: backend

namespace: attenza

spec:

replicas: 1

selector:


```
  matchLabels:

    app: backend

template:

  metadata:

    labels:

app: backend
```

B.3 Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

  name: backend-hpa

spec:

  minReplicas: 1

  maxReplicas: 8

  metrics:

    - type: Resource

      resource:

        name: cpu

        target:

          type: Utilization

          averageUtilization: 65
```

B.4 Ingress Configuration (Snippet)

```
apiVersion: networking.k8s.io/v1

kind: Ingress
```

metadata:

name: attenza-ingress

spec:

rules:

- host: yourdomain.com

Appendix C: Deployment Script

The deployment script automates the application of Kubernetes manifests in a defined sequence.

```
#!/bin/bash
```

```
kubectl apply -f namespace.yaml
```

```
kubectl apply -f mysql-secret.yaml
```

```
kubectl apply -f mysql.yaml
```

```
kubectl apply -f backend.yaml
```

```
kubectl apply -f frontend.yaml
```

```
kubectl apply -f ai-backend.yaml
```

```
kubectl apply -f ollama.yaml
```

```
kubectl apply -f cluster-issuer.yaml
```

```
kubectl apply -f ingress.yaml
```

```
echo "Deployment Completed Successfully"
```

This script ensures consistent and repeatable deployment.

Appendix D: Sample Code Snippets

This section includes selected code excerpts demonstrating core logic.

D.1 Authentication Controller (Spring Boot)

```
@PostMapping("/login")
```

```
public ResponseEntity<?> login(@RequestBody LoginRequest request) {  
    return ResponseEntity.ok(authService.authenticate(request));  
}
```

D.2 AI Backend Request Handling

```
response = requests.post(  
    OLLAMA_URL,  
    json={  
        "model": MODEL_NAME,  
        "messages": messages  
    }  
)
```

D.3 Attendance Submission Logic

```
public AttendanceSubmission submitAttendance(Long sessionId, Long studentId) {  
    AttendanceSubmission submission = new AttendanceSubmission();  
    submission.setSessionId(sessionId);  
    submission.setStudentId(studentId);  
    return attendanceRepository.save(submission);  
}
```